INSTITUTO POLITÉCTICO NACIONAL

UPIITA - Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas IPN

Base de Datos Distribuidas

Tarea 2 Arquitecturas para Bases de Datos Distribuidas

Marco González Luna 2024640044

Olvera Valdivia Cristobal 2024640014

Grupo

3TM2

Profesor

Carlos De La Cruz Sosa

upiita-ipn

Índice

TEMA 1: ARQUITECTURAS PARA BASES DE DATOS DISTRIBUIDAS 3

Perspectiva Principal: Özsu y Valduriez 3
Modelos Arquitectónicos según Özsu y Valduriez 4
Enfoques Modernos y de la Industria 5
Conclusión 6

TEMA 2: GESTORES DE BASES DE DATOS DISTRIBUIDAS 7

REFERENCIAS 10

Tema 1: Arquitecturas para bases de datos distribuidas

Perspectiva Principal: Özsu y Valduriez

Özsu y Valduriez definen la arquitectura de un sistema como la estructura de este: los componentes que lo forman, las funciones de cada uno y cómo interactúan entre sí.

Para clasificar y entender cualquier base de datos distribuida, los autores proponen un espacio multidimensional basado en **tres dimensiones principales**:

- **Distribución (Distribution):** Se refiere a si los componentes del sistema (datos y procesamiento) están ubicados en una misma máquina (centralizados) o dispersos a través de múltiples nodos conectados por una red (distribuidos).
- **Heterogeneidad (Heterogeneity):** Implica las diferencias tecnológicas en el sistema. Puede haber heterogeneidad de hardware, sistemas operativos o protocolos de red. Sin embargo, en bases de datos, la heterogeneidad más importante es la del modelo de datos (por ejemplo, mezclar una base de datos relacional con una orientada a objetos o NoSQL) y los lenguajes de consulta.
- **Autonomía (Autonomy):** Es el grado en que cada nodo o sistema de base de datos local puede operar de forma independiente. Se divide en:
    - *Autonomía de diseño:* Libertad para elegir el modelo de datos propio.
    - *Autonomía de comunicación:* Libertad para decidir con qué otros nodos comunicarse.
    - *Autonomía de ejecución:* Libertad para decidir cómo y cuándo ejecutar transacciones locales o globales.

Modelos Arquitectónicos según Özsu y Valduriez

Basándose en estas dimensiones, el libro destaca tres modelos arquitectónicos principales:

1. **Arquitectura Cliente-Servidor (Client-Server):** Es el enfoque más común. Existe una clara división de responsabilidades. Los **clientes** (o el front-end) manejan la interfaz de usuario, la formulación de consultas y la gestión de la sesión. Los **servidores** (back-end) se encargan de almacenar los datos, procesar las consultas, optimizarlas y controlar la concurrencia. Pueden existir arquitecturas de "Múltiples Clientes / Un Servidor" o "Múltiples Clientes / Múltiples Servidores".
2. **Arquitectura Peer-to-Peer (P2P / Totalmente Distribuida)**: En este modelo desaparece la distinción entre cliente y servidor. Cada nodo (o sitio) en la red tiene funcionalidades completas de un Sistema de Gestión de Bases de Datos (DBMS) y es considerado un "par" (peer). Cada nodo gestiona su propia base de datos local, pero también coopera con los demás para procesar transacciones globales.
3. **Sistemas Multibase de Datos (MDBS)**: Son arquitecturas diseñadas para integrar múltiples bases de datos que ya existían previamente. Suelen ser muy heterogéneas y altamente autónomas. Se dividen en:
    - No Federadas (Débilmente acopladas): No existe un esquema global central; los usuarios deben saber cómo conectarse a cada base de datos y unificar los datos por su cuenta.
    - Federadas (Fuertemente acopladas): Existe un esquema global integrado que unifica las vistas de todas las bases de datos locales, dando la apariencia de ser una sola.

Enfoques Modernos y de la Industria

1. **Arquitecturas por Recursos Compartidos (Clasificación de Hardware)**

Una forma muy común en la industria para catalogar la arquitectura es observar qué componentes físicos comparten los nodos:

- **Shared Memory (Memoria Compartida):** Múltiples procesadores comparten la misma memoria principal y los mismos discos. Es rápido, pero difícil de escalar.
- **Shared Disk (Disco Compartido):** Cada nodo tiene su propio procesador y memoria RAM, pero todos comparten el acceso a la red de almacenamiento (discos). Un ejemplo clásico es Oracle RAC.
- **Shared Nothing (Nada Compartido):** Cada nodo es completamente independiente, con su propio procesador, memoria y disco. Se comunican exclusivamente a través de la red. **Esta es la arquitectura dominante actual** para bases de datos distribuidas masivas (como Cassandra, CockroachDB o sistemas en la nube) porque ofrece la mejor escalabilidad horizontal.

2. **Arquitecturas Modernas de Consistencia y Replicación**

A nivel de software, las arquitecturas modernas (como las que describen blogs de ingeniería de software o documentación de bases de datos NoSQL/NewSQL) se centran en cómo se replican los datos:

- **Arquitectura Primario-Réplica (Master-Slave):** Un nodo actúa como primario y maneja todas las escrituras. Los nodos secundarios son réplicas que se utilizan únicamente para lecturas, aligerando la carga.
- **Arquitectura Multi-Master (Sin Maestro):** Todos los nodos son iguales y pueden procesar tanto lecturas como escrituras simultáneamente. Ofrece altísima disponibilidad, ya que si falla un centro de datos entero, los demás siguen procesando la información.
- **Base de Datos Particionada (Sharding):** La base de datos no se guarda entera en un solo servidor, sino que se divide lógicamente en "fragmentos" (shards). Cada servidor es responsable exclusivamente de una fracción de los datos.

Conclusión

En conclusión, el análisis de las arquitecturas para bases de datos distribuidas demuestra que estos sistemas han evolucionado de ser soluciones teóricas complejas a ser el pilar fundamental de las aplicaciones modernas. Los principios establecidos por Özsu y Valduriez siguen siendo el marco de referencia esencial para entender cómo un sistema equilibra la distribución de datos, la heterogeneidad tecnológica y la autonomía de sus nodos. Sin embargo, la necesidad actual de procesamiento a gran escala ha impulsado a la industria a adoptar principalmente arquitecturas del tipo *Shared Nothing* y modelos de replicación *Multi-Master*. En última instancia, el éxito de cualquier arquitectura distribuida hoy en día radica en su capacidad para ocultar toda esta complejidad subyacente, cumpliendo con el objetivo principal del sistema: ofrecer una transparencia total para que el usuario o la aplicación interactúen con la red como si se tratara de una única base de datos local.

Tema 2: Gestores de bases de datos distribuidas

Nombre del Gestor	Estrategia de Distribución (Fragmentación y Replicación)	Costo (Gratuito / De pago)	Tipo de Modelo de Datos
Oracle Database	**Fragmentación:** Horizontal nativa de alto rendimiento (*Oracle Sharding*). **Replicación:** Primario-Réplica (*Active Data Guard*) y Multi-Maestro/Bidireccional (*Oracle GoldenGate*).	**De pago** (Licencias comerciales Enterprise). Ofrece una versión limitada gratuita (*Express Edition - XE*).	**Relacional / Multimodelo** (SQL tradicional, JSON, XML, Grafos, Espacial).
Microsoft SQL Server	**Fragmentación:** Horizontal mediante particionamiento de tablas y vistas particionadas distribuidas. **Replicación:** Primario-Réplica (*Always On Availability Groups*) y Replicación de mezcla (*Merge Replication* para escenarios tipo Multi-Maestro).	**De pago** (Licencias comerciales Standard y Enterprise). Ofrece versiones gratuitas limitadas (*Express*) o para pruebas (*Developer*).	**Relacional / Multimodelo** (SQL tradicional, JSON, XML, Grafos).

Apache Cassandra	**Fragmentación:** Horizontal (*Consistent Hashing*). **Replicación:** Multi-Maestro (*Masterless* / Activo-Activo).	**Gratuito** (Open Source). Existen versiones de pago con soporte empresarial (ej. DataStax).	**NoSQL** (Almacén de columnas anchas / *Wide-column*).
CockroachDB	**Fragmentación:** Horizontal automatizada por rangos (*Ranges*). **Replicación:** Multi-Maestro (mediante algoritmo de consenso *Raft*).	**Mixto:** Versión *Core* gratuita (Open Source). Versiones *Enterprise* y *Cloud* de pago.	**Relacional Distribuido** (NewSQL / SQL con garantías ACID).
MongoDB	**Fragmentación:** Horizontal (*Sharding* basado en claves de fragmentación). **Replicación:** Primario-Réplica (*Replica Sets*).	**Mixto:** *Community Edition* gratuita. *MongoDB Atlas* (Nube) y *Enterprise Advanced* de pago.	**NoSQL** (Orientado a Documentos / JSON-BSON).
Amazon DynamoDB	**Fragmentación:** Horizontal automática (basada en *Partition Keys*). **Replicación:** Multi-Maestro y Multi-Región (*Global Tables*)	**De pago** (Modelo de pago por uso en AWS). Incluye una capa gratuita mensual limitada.	**NoSQL** (Clave-Valor y Orientado a Documentos).

PostgreSQL (con extensión Citus)	**Fragmentación:** Horizontal (mediante tablas distribuidas). **Replicación:** Primario-Réplica (estándar), soportando múltiples nodos.	**Gratuito** (Citus ahora es completamente Open Source).	**Relacional** (Objeto-Relacional / SQL clásico).
MySQL Cluster	**Fragmentación:** Horizontal automática (*Auto-sharding*). **Replicación:** Multi-Maestro síncrona (todos los nodos pueden escribir).	**Gratuito** (GPL). Versión *Commercial* de pago con soporte de Oracle.	**Relacional** (SQL tradicional).

Referencias

Architecture overview. (s/f). CockroachDB Docs. Recuperado el 12 de marzo de 2026, de https://www.cockroachlabs.com/docs/stable/architecture/overview.html

(S/f-a). Dropbox.com. Recuperado el 12 de marzo de 2026, de https://www.dropbox.com/scl/fi/wkxg584wnnu77b6iq7dbj/M.-Tamer-zsu-Patrick-Valduriez-Principles-of-Distributed-Database-Systems-Third-Edition-Springer-2011.pdf?rlkey=aq76sccqh7qbce46iow95ycd7&e=2&st=glp271ca&dl=0

(S/f-b). Amazon.com. Recuperado el 12 de marzo de 2026, de https://docs.aws.amazon.com/es_es/aurora-dsql/latest/userguide/what-is-aurora-dsql.html

Apache Cassandra documentation. (s/f). Apache Cassandra. Recuperado el 16 de marzo de 2026, de https://cassandra.apache.org/doc/latest/cassandra/architecture/index.html

Architecture overview. (s/f). CockroachDB Docs. Recuperado el 16 de marzo de 2026, de https://www.cockroachlabs.com/docs/stable/architecture/overview.html

Citus documentation - Citus 13.0.1 documentation. (s/f). Citusdata.com. Recuperado el 16 de marzo de 2026, de https://docs.citusdata.com/en/stable/

Globally Distributed AI Database. (s/f). Oracle.com. Recuperado el 16 de marzo de 2026, de https://www.oracle.com/database/technologies/high-availability/sharding.html

High availability database - HA solutions. (s/f). Oracle.com. Recuperado el 16 de marzo de 2026, de https://www.oracle.com/database/technologies/high-availability.html

MashaMSFT. (s/f). *¿Qué es un grupo de disponibilidad AlwaysOn?* Microsoft.com. Recuperado el 16 de marzo de 2026, de https://learn.microsoft.com/es-es/sql/database-engine/availability-groups/windows/always-on -availability-groups-sql-server

Replication. (s/f). Mongodb.com. Recuperado el 16 de marzo de 2026, de https://www.mongodb.com/docs/manual/replication/

Sharding. (s/f). Mongodb.com. Recuperado el 16 de marzo de 2026, de https://www.mongodb.com/docs/manual/sharding/

VanMSFT. (s/f). *Tablas e índices con particiones*. Microsoft.com. Recuperado el 16 de marzo de 2026, de https://learn.microsoft.com/es-es/sql/relational-databases/partitions/partitioned-tables-and-indexes

(S/f-a). Amazon.com. Recuperado el 16 de marzo de 2026, de https://docs.aws.amazon.com/es_es/amazondynamodb/latest/developerguide/HowItWorks.html

(S/f-b). Mysql.com. Recuperado el 16 de marzo de 2026, de https://dev.mysql.com/doc/refman/8.0/en/mysql-cluster.html